

TP_2

202111052 한민준

202110940 신준우



Ablation Study

1 .global _main_init	.global _main_init	.global _main_init
2 .text	.text	.text
3 lui s0, 0x00000	lui s0, 0x00000	lui s0, 0x00000
4 lui s1, 0x00000	lui s1, 0x00000	lui s1, 0x00000
5 addi s1, s1, 0x400	addi s1, s1, 0x400	addi s1, s1, 0x400
6 lui s2, 0x01000	lui s2, 0x01000	lui s2, 0x01000
7 lui s3, 0x03000	lui s3, 0x03000	lui s3, 0x03000
8 lui s4, 0x05000	lui s4, 0x05000	lui s4, 0x05000
9 lui s5, 0x06000	lui s5, 0x06000	lui s5, 0x06000
10 lui s6, 0x07000	lui s6, 0x07000	lui s6, 0x07000
11 lw s7, 0(s3)	delay	delay
12 lw t0, 0(s0) # t0 = SRAM_IO[0]	lw s7, 0(s3)	delay
13 lw t1, 128(s0) # t1 = SRAM_IO[32]	delay	lw s7, 0(s3)
14 sw t0, 0(s2) # SRAM_O[0] = SRAM_IO[0]	lw t0, 0(s0) # t0 = SRAM_IO[0]	delay
15 sw t1, 4(s2) # SRAM_O[1] = SRAM_IO[32]	lw t1, 128(s0) # t1 = SRAM_IO[32]	delay
16 lw t4, 0(s3) # t4 = SRAM_W[0]	twiddle preload for stage_1~4	lw t0, 0(s0) # t0 = SRAM_IO[0]
17 lw t2, 0(s2) # t2 = SRAM_O[0]	twiddle preload for stage_1~4	lw t1, 128(s0) # t1 = SRAM_IO[32]
18 lw t3, 4(s2) # t3 = SRAM_O[1]	twiddle preload for stage_1~4	delay
19 sw t2, 0(s4) # dpath_reg_0 = SRAM_O[0]	sw t0, 8(s2) # SRAM_O[2] = SRAM_IO[16] (preloaded)	twiddle preload for stage_1~4
20 sw t3, 0(s5) # dpath_reg_1 = SRAM_O[1]	sw t1, 12(s2) # SRAM_O[3] = SRAM_IO[48] (preloaded)	twiddle preload for stage_1~4
21 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	twiddle preload for stage_1~4
22 twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_O[2]	delay
23 lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 12(s2) # t3 = SRAM_O[3]	delay
24 lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_O[2]	sw t0, 8(s2) # SRAM_O[2] = SRAM_IO[16] (preloaded)
25 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_O[3]	sw t1, 12(s2) # SRAM_O[3] = SRAM_IO[48] (preloaded)
26 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
27 sw t2, 0(s2) # SRAM_O[0] = X	twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_O[2]
28 sw t3, 4(s2) # SRAM_O[1] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_O[3]
29 sw t0, 8(s2) # SRAM_O[2] = SRAM_IO[16] (preloaded)	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_O[2]
30 sw t1, 12(s2) # SRAM_O[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_O[3]
31 lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
32 lw t2, 8(s2) # t2 = SRAM_O[2]	sw t2, 8(s2) # SRAM_O[2] = X	twiddle preload for stage_1~4
33 lw t3, 12(s2) # t3 = SRAM_O[3]	sw t3, 12(s2) # SRAM_O[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
34 sw t2, 0(s4) # dpath_reg_0 = SRAM_O[2]	sw t0, 8(s2) # SRAM_O[2] = SRAM_IO[16] (preloaded)	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
35 sw t3, 0(s5) # dpath_reg_1 = SRAM_O[3]	sw t1, 12(s2) # SRAM_O[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
36 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
37 twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_O[2]	sw t2, 8(s2) # SRAM_O[2] = X
38 lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_O[3]	sw t3, 12(s2) # SRAM_O[3] = Y
39 lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_O[2]	sw t0, 8(s2) # SRAM_O[2] = SRAM_IO[16] (preloaded)
40 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_O[3]	sw t1, 12(s2) # SRAM_O[3] = SRAM_IO[48] (preloaded)
41 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
42 sw t2, 8(s2) # SRAM_O[2] = X	twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_O[2]
43 sw t3, 12(s2) # SRAM_O[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_O[3]
44 ~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_O[2]
45 ~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_O[3]
46 ~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
47 ~	sw t2, 8(s2) # SRAM_O[2] = X	twiddle preload for stage_1~4
48 ~	sw t3, 12(s2) # SRAM_O[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
49 ~	~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
50 ~	~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
51 ~	~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
52 ~	~	sw t2, 8(s2) # SRAM_O[2] = X
53 ~	~	sw t3, 12(s2) # SRAM_O[3] = Y

Idea 1

core_0,1,2 총 3개를 이용하여
각 set에 대해 병렬화

core0	core1	core2
1 .global _main_init	.global _main_init	.global _main_init
2 .text	.text	.text
3 lui s0, 0x00000	lui s0, 0x00000	lui s0, 0x00000
4 lui s1, 0x00000	lui s1, 0x00000	lui s1, 0x00000
5 delay	addi s0, s0, 256	addi s0, s0, 512
6 delay	addi s1, s1, 256	addi s1, s1, 512
7 addi s1, s1, 0x400	addi s1, s1, 0x400	addi s1, s1, 0x400
8 lui s2, 0x01000	lui s2, 0x02000	lui s2, 0x04000
9 lui s3, 0x03000	lui s3, 0x03000	lui s3, 0x03000
10 lui s4, 0x05000	lui s4, 0x05000	lui s4, 0x05000
11 lui s5, 0x06000	lui s5, 0x06000	lui s5, 0x06000
12 lui s6, 0x07000	lui s6, 0x07000	lui s6, 0x07000
13 lw s7, 0(s3)	delay	delay
14 lw t0, 0(s0) # t0 = SRAM_IO[0]	lw s7, 0(s3)	delay
15 lw t1, 128(s0) # t1 = SRAM_IO[32]	delay	lw s7, 0(s3)
16 sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	lw t0, 0(s0) # t0 = SRAM_IO[0]	delay
17 sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	lw t1, 128(s0) # t1 = SRAM_IO[32]	delay
18 lw t4, 0(s3) # t4 = SRAM_W[0]	delay	lw t0, 0(s0) # t0 = SRAM_IO[0]
19 lw t2, 0(s2) # t2 = SRAM_0[0]	delay	lw t1, 128(s0) # t1 = SRAM_IO[32]
20 lw t3, 4(s2) # t3 = SRAM_0[1]	delay	delay
21 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	delay
22 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	delay
23 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	delay
24 load twiddle factor for stage_1~4	lw t2, 0(s2) # t2 = SRAM_0[0]	delay
25 lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 4(s2) # t3 = SRAM_0[1]	delay
26 lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]
27 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]
28 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
29 sw t2, 0(s2) # SRAM_0[0] = X	load twiddle factor for stage_1~4	lw t2, 0(s2) # t2 = SRAM_0[0]
30 sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 4(s2) # t3 = SRAM_0[1]
31 sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]
32 sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]
33 lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
34 lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 0(s2) # SRAM_0[0] = X	load twiddle factor for stage_1~4
35 lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1
36 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1
37 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
38 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
39 load twiddle factor for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 0(s2) # SRAM_0[0] = X
40 lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 4(s2) # SRAM_0[1] = Y
41 lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
42 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
43 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
44 sw t2, 8(s2) # SRAM_0[2] = X	load twiddle factor for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]
45 sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]
46 ~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
47 ~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
48 ~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
49 ~	sw t2, 8(s2) # SRAM_0[2] = X	load twiddle factor for stage_1~4
50 ~	sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
51 ~	~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
52 ~	~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
53 ~	~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
54 ~	~	sw t2, 8(s2) # SRAM_0[2] = X
55 ~	~	sw t3, 12(s2) # SRAM_0[3] = Y

Idea 2

한 bf의 clk-cycle 수를 3의 배수로 맞추므로써, SRAM의 1R1W 관리

	core0	core1	core2
1	.global _main_init	.global _main_init	.global _main_init
2	.text	.text	.text
3	lui s0, 0x00000	lui s0, 0x00000	lui s0, 0x00000
4	lui s1, 0x00000	lui s1, 0x00000	lui s1, 0x00000
5	delay	addi s0, s0, 256	addi s0, s0, 512
6	delay	addi s1, s1, 256	addi s1, s1, 512
7	addi s1, s1, 0x400	addi s1, s1, 0x400	addi s1, s1, 0x400
8	lui s2, 0x01000	lui s2, 0x02000	lui s2, 0x04000
9	lui s3, 0x03000	lui s3, 0x03000	lui s3, 0x03000
10	lui s4, 0x05000	lui s4, 0x05000	lui s4, 0x05000
11	lui s5, 0x06000	lui s5, 0x06000	lui s5, 0x06000
12	lui s6, 0x07000	lui s6, 0x07000	lui s6, 0x07000
13	lw s7, 0(s3)	delay	delay
14	lw t0, 0(s0) # t0 = SRAM_IO[0]	lw s7, 0(s3)	delay
15	lw t1, 128(s0) # t1 = SRAM_IO[32]	delay	lw s7, 0(s3)
16	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	lw t0, 0(s0) # t0 = SRAM_IO[0]	delay
17	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	lw t1, 128(s0) # t1 = SRAM_IO[32]	delay
18	lw t4, 0(s3) # t4 = SRAM_W[0]	delay	lw t0, 0(s0) # t0 = SRAM_IO[0]
19	lw t2, 0(s2) # t2 = SRAM_0[0]	delay	lw t1, 128(s0) # t1 = SRAM_IO[32]
20	lw t3, 4(s2) # t3 = SRAM_0[1]	delay	delay
21	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	delay
22	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	delay
23	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	delay
24	load twiddle factor for stage_1~4	lw t2, 0(s2) # t2 = SRAM_0[0]	delay
25	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 4(s2) # t3 = SRAM_0[1]	delay
26	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]
27	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]
28	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
29	sw t2, 0(s2) # SRAM_0[0] = X	load twiddle factor for stage_1~4	lw t2, 0(s2) # t2 = SRAM_0[0]
30	sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 4(s2) # t3 = SRAM_0[1]
31	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]
32	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]
33	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
34	lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 0(s2) # SRAM_0[0] = X	load twiddle factor for stage_1~4
35	lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1
36	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1
37	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
38	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
39	load twiddle factor for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 0(s2) # SRAM_0[0] = X
40	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 4(s2) # SRAM_0[1] = Y
41	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
42	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
43	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
44	sw t2, 8(s2) # SRAM_0[2] = X	load twiddle factor for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]
45	sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]
46	~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
47	~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
48	~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
49	~	sw t2, 8(s2) # SRAM_0[2] = X	load twiddle factor for stage_1~4
50	~	sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
51	~	~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
52	~	~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
53	~	~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
54	~	~	sw t2, 8(s2) # SRAM_0[2] = X
55	~	~	sw t3, 12(s2) # SRAM_0[3] = Y

Idea 3

STAGE_0에서 clk-cycle의 개수를 3의 배수(15)로 맞추되,
delay시간을 활용하기 위하여 다음 bf에 사용되는
input을 미리 load

core0	core1	core2
1 .global _main_init	.global _main_init	.global _main_init
2 .text	.text	.text
3 lui s0, 0x00000	lui s0, 0x00000	lui s0, 0x00000
4 lui s1, 0x00000	lui s1, 0x00000	lui s1, 0x00000
5 delay	addi s0, s0, 256	addi s0, s0, 512
6 delay	addi s1, s1, 256	addi s1, s1, 512
7 addi s1, s1, 0x400	addi s1, s1, 0x400	addi s1, s1, 0x400
8 lui s2, 0x01000	lui s2, 0x02000	lui s2, 0x04000
9 lui s3, 0x03000	lui s3, 0x03000	lui s3, 0x03000
10 lui s4, 0x05000	lui s4, 0x05000	lui s4, 0x05000
11 lui s5, 0x06000	lui s5, 0x06000	lui s5, 0x06000
12 lui s6, 0x07000	lui s6, 0x07000	lui s6, 0x07000
13 lw s7, 0(s3)	delay	delay
14 lw t0, 0(s0) # t0 = SRAM_IO[0]	lw s7, 0(s3)	delay
15 lw t1, 128(s0) # t1 = SRAM_IO[32]	delay	lw s7, 0(s3)
16 sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	lw t0, 0(s0) # t0 = SRAM_IO[0]	delay
17 sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	lw t1, 128(s0) # t1 = SRAM_IO[32]	delay
18 lw t4, 0(s3) # t4 = SRAM_W[0]	delay	lw t0, 0(s0) # t0 = SRAM_IO[0]
19 lw t2, 0(s2) # t2 = SRAM_0[0]	delay	lw t1, 128(s0) # t1 = SRAM_IO[32]
20 lw t3, 4(s2) # t3 = SRAM_0[1]	delay	delay
21 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	delay
22 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	delay
23 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	delay
24 load twiddle factor for stage 1~4	lw t2, 0(s2) # t2 = SRAM_0[0]	delay
25 lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 4(s2) # t3 = SRAM_0[1]	delay
26 lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]
27 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]
28 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
29 sw t2, 0(s2) # SRAM_0[0] = X	load twiddle factor for stage 1~4	lw t2, 0(s2) # t2 = SRAM_0[0]
30 sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 4(s2) # t3 = SRAM_0[1]
31 sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]
32 sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]
33 lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
34 lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 0(s2) # SRAM_0[0] = X	load twiddle factor for stage 1~4
35 lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1
36 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1
37 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
38 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
39 load twiddle factor for stage 1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 0(s2) # SRAM_0[0] = X
40 lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 4(s2) # SRAM_0[1] = Y
41 lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
42 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
43 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
44 sw t2, 8(s2) # SRAM_0[2] = X	load twiddle factor for stage 1~4	lw t2, 8(s2) # t2 = SRAM_0[2]
45 sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]
46 ~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
47 ~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
48 ~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
49 ~	sw t2, 8(s2) # SRAM_0[2] = X	load twiddle factor for stage 1~4
50 ~	sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
51 ~	~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
52 ~	~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
53 ~	~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
54 ~	~	sw t2, 8(s2) # SRAM_0[2] = X
55 ~	~	sw t3, 12(s2) # SRAM_0[3] = Y

Idea 4

STAGE_0의 delay 시간을 활용하여 STAGE_1~4에
서 사용하는 twiddle factor 16개를 전부 lw

core0	core1	core2
1 .global _main_init	.global _main_init	.global _main_init
2 .text	.text	.text
3 lui s0, 0x00000	lui s0, 0x00000	lui s0, 0x00000
4 lui s1, 0x00000	lui s1, 0x00000	lui s1, 0x00000
5 delay	addi s0, s0, 256	addi s0, s0, 512
6 delay	addi s1, s1, 256	addi s1, s1, 512
7 addi s1, s1, 0x400	addi s1, s1, 0x400	addi s1, s1, 0x400
8 lui s2, 0x01000	lui s2, 0x02000	lui s2, 0x04000
9 lui s3, 0x03000	lui s3, 0x03000	lui s3, 0x03000
10 lui s4, 0x05000	lui s4, 0x05000	lui s4, 0x05000
11 lui s5, 0x06000	lui s5, 0x06000	lui s5, 0x06000
12 lui s6, 0x07000	lui s6, 0x07000	lui s6, 0x07000
13 lw s7, 0(s3)	delay	delay
14 lw t0, 0(s0) # t0 = SRAM_IO[0]	lw s7, 0(s3)	delay
15 lw t1, 128(s0) # t1 = SRAM_IO[32]	delay	lw s7, 0(s3)
16 sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	lw t0, 0(s0) # t0 = SRAM_IO[0]	delay
17 sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	lw t1, 128(s0) # t1 = SRAM_IO[32]	delay
18 lw t4, 0(s3) # t4 = SRAM_W[0]	delay	lw t0, 0(s0) # t0 = SRAM_IO[0]
19 lw t2, 0(s2) # t2 = SRAM_0[0]	delay	lw t1, 128(s0) # t1 = SRAM_IO[32]
20 lw t3, 4(s2) # t3 = SRAM_0[1]	delay	delay
21 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	delay
22 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	delay
23 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	delay
24 load twiddle factor for stage 1~4	lw t2, 0(s2) # t2 = SRAM_0[0]	delay
25 lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 4(s2) # t3 = SRAM_0[1]	delay
26 lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t0, 0(s2) # dpath_reg_0 = SRAM_0[0]	sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]
27 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]
28 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
29 sw t2, 0(s2) # SRAM_0[0] = X	load twiddle factor for stage 1~4	lw t2, 0(s2) # t2 = SRAM_0[0]
30 sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 4(s2) # t3 = SRAM_0[1]
31 sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]
32 sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]
33 lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
34 lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 0(s2) # SRAM_0[0] = X	load twiddle factor for stage 1~4
35 lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1
36 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1
37 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
38 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
39 load twiddle factor for stage 1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 0(s2) # SRAM_0[0] = X
40 lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 4(s2) # SRAM_0[1] = Y
41 lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
42 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
43 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
44 sw t2, 8(s2) # SRAM_0[2] = X	load twiddle factor for stage 1~4	lw t2, 8(s2) # t2 = SRAM_0[2]
45 sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]
46 ~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
47 ~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
48 ~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
49 ~	sw t2, 8(s2) # SRAM_0[2] = X	load twiddle factor for stage 1~4
50 ~	sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
51 ~	~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
52 ~	~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
53 ~	~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
54 ~	~	sw t2, 8(s2) # SRAM_0[2] = X
55 ~	~	sw t3, 12(s2) # SRAM_0[3] = Y

Idea 5

STATE_1~4에서, 1개의 bf의 clk-cycle수를 9로 맞추기 위해, dpath_reg의 연산시간에 다음 bf의 input에 대한 preload를 진행함으로써, 낭비되는 시간 제거

95 lw t0, 0(s2) # load SRAM0[0]	sw t3, 252(s2) # SRAM_0[63] = Y	delay
96 lw t1, 8(s2) # load SRAM0[2]	delay	delay
97 sw t0, 0(s4) # store to datapath input 0	delay	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
98 sw t1, 0(s5) # store to datapath input 1	lw t0, 0(s2) # load SRAM0[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
99 sw s7, 0(s6) # store twiddle factor W[0]	lw t1, 8(s2) # load SRAM0[2]	sw t2, 248(s2) # SRAM_0[62] = X
100 lw s8, 16(s2) # preload SRAM0[4] for next butterfly	sw t0, 0(s4) # store to datapath input 0	sw t3, 252(s2) # SRAM_0[63] = Y
101 lw t2, 4(s4) # load result X from datapath	sw t1, 0(s5) # store to datapath input 1	lw t0, 0(s2) # load SRAM0[0]
102 lw t3, 4(s5) # load result Y from datapath	sw s7, 0(s6) # store twiddle factor W[0]	lw t1, 8(s2) # load SRAM0[2]
103 sw t2, 0(s2) # store X to SRAM0[0]	lw s8, 16(s2) # preload SRAM0[4] for next butterfly	sw t0, 0(s4) # store to datapath input 0
104 sw t3, 8(s2) # store Y to SRAM0[2]	lw t2, 4(s4) # load result X from datapath	sw t1, 0(s5) # store to datapath input 1
105 lw t1, 24(s2) # load SRAM0[6]	lw t3, 4(s5) # load result Y from datapath	sw s7, 0(s6) # store twiddle factor W[0]
106 sw s8, 0(s4) # store preloaded SRAM0[4]	sw t2, 0(s2) # store X to SRAM0[0]	lw s8, 16(s2) # preload SRAM0[4] for next butterfly
107 sw t1, 0(s5)	sw t3, 8(s2) # store Y to SRAM0[2]	lw t2, 4(s4) # load result X from datapath
108 sw s7, 0(s6)	lw t1, 24(s2) # load SRAM0[6]	lw t3, 4(s5) # load result Y from datapath
109 lw s8, 32(s2) # preload SRAM0[8]	sw s8, 0(s4) # store preloaded SRAM0[4]	sw t2, 0(s2) # store X to SRAM0[0]
110 lw t2, 4(s4)	sw t1, 0(s5)	sw t3, 8(s2) # store Y to SRAM0[2]
111 lw t3, 4(s5)	sw s7, 0(s6)	lw t1, 24(s2) # load SRAM0[6]
112 sw t2, 16(s2) # store to SRAM0[4]	lw s8, 32(s2) # preload SRAM0[8]	sw s8, 0(s4) # store preloaded SRAM0[4]
113 sw t3, 24(s2) # store to SRAM0[6]	lw t2, 4(s4)	sw t1, 0(s5)
114 ~	lw t3, 4(s5)	sw s7, 0(s6)
115 ~	sw t2, 16(s2) # store to SRAM0[4]	lw s8, 32(s2) # preload SRAM0[8]
116 ~	sw t3, 24(s2) # store to SRAM0[6]	lw t2, 4(s4)
117 ~	~	lw t3, 4(s5)
118 ~	~	sw t2, 16(s2) # store to SRAM0[4]
119 ~	~	sw t3, 24(s2) # store to SRAM0[6]
120 ~	~	~
121 ~	~	~
122 ~	~	~
123 lw t1, 124(s2) # SRAM0[31]	~	~
124 sw s8, 0(s4)	~	~
125 sw t1, 0(s5)	~	~
126 sw gp, 0(s6)	lw t1, 124(s2) # SRAM0[31]	~
127 lw s8, 188(s2) # preload SRAM0[47]	sw s8, 0(s4)	~
128 lw t2, 4(s4)	sw t1, 0(s5)	~
129 lw t3, 4(s5)	sw gp, 0(s6)	lw t1, 124(s2) # SRAM0[31]
130 sw t2, 60(s2)	lw s8, 188(s2) # preload SRAM0[47]	sw s8, 0(s4)
131 sw t3, 124(s2)	lw t2, 4(s4)	sw t1, 0(s5)
132 lw t1, 252(s2) # SRAM0[63]	lw t3, 4(s5)	sw gp, 0(s6)
133 sw s8, 0(s4) # SRAM0[47]	sw t2, 60(s2)	lw s8, 188(s2) # preload SRAM0[47]
134 sw t1, 0(s5)	sw t3, 124(s2)	lw t2, 4(s4)
135 sw gp, 0(s6)	lw t1, 252(s2) # SRAM0[63]	lw t3, 4(s5)
136 delay	sw s8, 0(s4) # SRAM0[47]	sw t2, 60(s2)
137 lw t2, 4(s4)	sw t1, 0(s5)	sw t3, 124(s2)

소요시간은 작지만,
SRAM occupancy가 매우 높다는 문제 발생

```
Validating bit-accurate output...  
  - Total mismatches: 0 / 192  
Checking SRAM occupancy...  
  - SRAM 2: occupancy = 64  
  - SRAM 3: occupancy = 64  
  - SRAM 5: occupancy = 64  
Test passed. Ending simulation.  
$finish called from file "../..tb/tb_mem_IO.v", line 132.  
$finish at simulation time 4394000  
Simulation complete, time is 4394000 ps.
```


Idea 6

SRAM occupancy를 192 → 64로 줄이기 위해
한 개의 set를 3개의 core가 동시에 bf연산 수행하고,
각 stage의 중간결과를 SRAM_0에 저장

1 .global _main_init	.global _main_init	.global _main_init
2 .text	.text	.text
3 lui s0, 0x00000	lui s0, 0x00000	lui s0, 0x00000
4 lui s1, 0x00000	lui s1, 0x00000	lui s1, 0x00000
5 addi s1, s1, 0x400	addi s1, s1, 0x400	addi s1, s1, 0x400
6 lui s2, 0x01000	lui s2, 0x01000	lui s2, 0x01000
7 lui s3, 0x03000	lui s3, 0x03000	lui s3, 0x03000
8 lui s4, 0x05000	lui s4, 0x05000	lui s4, 0x05000
9 lui s5, 0x06000	lui s5, 0x06000	lui s5, 0x06000
10 lui s6, 0x07000	lui s6, 0x07000	lui s6, 0x07000
11 lw s7, 0(s3)	delay	delay
12 lw t0, 0(s0) # t0 = SRAM_IO[0]	lw s7, 0(s3)	delay
13 lw t1, 128(s0) # t1 = SRAM_IO[32]	delay	lw s7, 0(s3)
14 sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	lw t0, 0(s0) # t0 = SRAM_IO[0]	delay
15 sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	lw t1, 128(s0) # t1 = SRAM_IO[32]	delay
16 lw t4, 0(s3) # t4 = SRAM_W[0]	twiddle preload for stage_1~4	lw t0, 0(s0) # t0 = SRAM_IO[0]
17 lw t2, 0(s2) # t2 = SRAM_0[0]	twiddle preload for stage_1~4	lw t1, 128(s0) # t1 = SRAM_IO[32]
18 lw t3, 4(s2) # t3 = SRAM_0[1]	twiddle preload for stage_1~4	delay
19 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	twiddle preload for stage_1~4
20 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	twiddle preload for stage_1~4
21 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	twiddle preload for stage_1~4
22 twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	delay
23 lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 12(s2) # t3 = SRAM_0[3]	delay
24 lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
25 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
26 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
27 sw t2, 0(s2) # SRAM_0[0] = X	twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]
28 sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]
29 sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
30 sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
31 lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
32 lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 8(s2) # SRAM_0[2] = X	twiddle preload for stage_1~4
33 lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
34 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
35 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
36 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
37 twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 8(s2) # SRAM_0[2] = X
38 lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 12(s2) # SRAM_0[3] = Y
39 lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
40 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
41 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
42 sw t2, 8(s2) # SRAM_0[2] = X	twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]
43 sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]
44 ~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
45 ~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
46 ~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
47 ~	sw t2, 8(s2) # SRAM_0[2] = X	twiddle preload for stage_1~4
48 ~	sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
49 ~	~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
50 ~	~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
51 ~	~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
52 ~	~	sw t2, 8(s2) # SRAM_0[2] = X
53 ~	~	sw t3, 12(s2) # SRAM_0[3] = Y

Idea 7

1개의 stage(32개 bf)를 11, 11, 10개씩
분담하여 수행

1.global_main_init	global_main_init	global_main_init
2.text	.text	.text
3 lui s0, 0x00000	lui s0, 0x00000	lui s0, 0x00000
4 lui s1, 0x00000	lui s1, 0x00000	lui s1, 0x00000
5 addi s1, s1, 0x400	addi s1, s1, 0x400	addi s1, s1, 0x400
6 lui s2, 0x01000	lui s2, 0x01000	lui s2, 0x01000
7 lui s3, 0x03000	lui s3, 0x03000	lui s3, 0x03000
8 lui s4, 0x05000	lui s4, 0x05000	lui s4, 0x05000
9 lui s5, 0x06000	lui s5, 0x06000	lui s5, 0x06000
10 lui s6, 0x07000	lui s6, 0x07000	lui s6, 0x07000
11 lw s7, 0(s3)	delay	delay
12 lw t0, 0(s0) # t0 = SRAM_IO[0]	delay	delay
13 lw t1, 128(s0) # t1 = SRAM_IO[32]	delay	delay
14 sw t0, 0(s2) # SRAM_0[0] = SRAM_IO[0]	lw t0, 0(s0) # t0 = SRAM_IO[0]	lw t0, 0(s0) # t0 = SRAM_IO[0]
15 sw t1, 4(s2) # SRAM_0[1] = SRAM_IO[32]	lw t1, 128(s0) # t1 = SRAM_IO[32]	lw t1, 128(s0) # t1 = SRAM_IO[32]
16 lw t4, 0(s3) # t4 = SRAM_W[0]	twiddle preload for stage_1~4	twiddle preload for stage_1~4
17 lw t2, 0(s2) # t2 = SRAM_0[0]	twiddle preload for stage_1~4	twiddle preload for stage_1~4
18 lw t3, 4(s2) # t3 = SRAM_0[1]	twiddle preload for stage_1~4	twiddle preload for stage_1~4
19 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[0]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
20 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[1]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
21 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
22 twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	lw t2, 8(s2) # t2 = SRAM_0[2]
23 lw t0, 64(s0) # Preload t0 = SRAM_IO[16] for Butterfly 1	lw t3, 12(s2) # t3 = SRAM_0[3]	lw t3, 12(s2) # t3 = SRAM_0[3]
24 lw t1, 192(s0) # Preload t1 = SRAM_IO[48] for Butterfly 1	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
25 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
26 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	sw t4, 0(s6) # dpath_reg_2 = W[0]
27 sw t2, 0(s2) # SRAM_0[0] = X	twiddle preload for stage_1~4	twiddle preload for stage_1~4
28 sw t3, 4(s2) # SRAM_0[1] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
29 sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
30 sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
31 lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
32 lw t2, 8(s2) # t2 = SRAM_0[2]	sw t2, 8(s2) # SRAM_0[2] = X	sw t2, 8(s2) # SRAM_0[2] = X
33 lw t3, 12(s2) # t3 = SRAM_0[3]	sw t3, 12(s2) # SRAM_0[3] = Y	sw t3, 12(s2) # SRAM_0[3] = Y
34 sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
35 sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
36 sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
37 twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	lw t2, 8(s2) # t2 = SRAM_0[2]
38 lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]	lw t3, 12(s2) # t3 = SRAM_0[3]
39 lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
40 lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
41 lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	sw t4, 0(s6) # dpath_reg_2 = W[0]
42 sw t2, 8(s2) # SRAM_0[2] = X	twiddle preload for stage_1~4	twiddle preload for stage_1~4
43 sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2
44 ~	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2
45 ~	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
46 ~	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
47 ~	sw t2, 8(s2) # SRAM_0[2] = X	sw t2, 8(s2) # SRAM_0[2] = X
48 ~	sw t3, 12(s2) # SRAM_0[3] = Y	sw t3, 12(s2) # SRAM_0[3] = Y
49 ~	~	~
50 ~	~	~
51 ~	~	~
52 ~	~	~
53 ~	~	~

Idea 8

STAGE_0에서 1로, STAGE_4에서 5로 넘어갈 때
각 코어의 싱크를 맞추기 위해 delay삽입(add x0, x0, x0)

58	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	~	~
59	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	~	~
60	lw t4, 0(s3) # t4 = SRAM_W[0]	~	~
61	lw t2, 8(s2) # t2 = SRAM_0[2]	~	~
62	lw t3, 12(s2) # t3 = SRAM_0[3]	~	~
63	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)	~
64	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)	~
65	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	~
66	twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]	~
67	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]	~
68	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]	sw t0, 8(s2) # SRAM_0[2] = SRAM_IO[16] (preloaded)
69	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]	sw t1, 12(s2) # SRAM_0[3] = SRAM_IO[48] (preloaded)
70	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]
71	sw t2, 8(s2) # SRAM_0[2] = X	twiddle preload for stage_1~4	lw t2, 8(s2) # t2 = SRAM_0[2]
72	sw t3, 12(s2) # SRAM_0[3] = Y	lw t0, 32(s0) # Preload t0 = SRAM_IO[8] for Butterfly 2	lw t3, 12(s2) # t3 = SRAM_0[3]
73	sw t0, 248(s2) # SRAM_0[62] = SRAM_IO[31] (preloaded)	lw t1, 160(s0) # Preload t1 = SRAM_IO[40] for Butterfly 2	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[2]
74	sw t1, 252(s2) # SRAM_0[63] = SRAM_IO[63] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[3]
75	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]
76	lw t2, 248(s2) # t2 = SRAM_0[62]	sw t2, 8(s2) # SRAM_0[2] = X	twiddle preload for stage_1~4
77	lw t3, 252(s2) # t3 = SRAM_0[63]	sw t3, 12(s2) # SRAM_0[3] = Y	twiddle preload for stage_1~4
78	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[62]	sw t0, 248(s2) # SRAM_0[62] = SRAM_IO[31] (preloaded)	twiddle preload for stage_1~4
79	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[63]	sw t1, 252(s2) # SRAM_0[63] = SRAM_IO[63] (preloaded)	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)
80	sw t4, 0(s6) # dpath_reg_2 = W[0]	lw t4, 0(s3) # t4 = SRAM_W[0]	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)
81	twiddle preload for stage_1~4	lw t2, 248(s2) # t2 = SRAM_0[62]	sw t2, 8(s2) # SRAM_0[2] = X
82	twiddle preload for stage_1~4	lw t3, 252(s2) # t3 = SRAM_0[63]	sw t3, 12(s2) # SRAM_0[3] = Y
83	twiddle preload for stage_1~4	sw t2, 0(s4) # dpath_reg_0 = SRAM_0[62]	delay
84	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	sw t3, 0(s5) # dpath_reg_1 = SRAM_0[63]	delay
85	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t4, 0(s6) # dpath_reg_2 = W[0]	delay
86	sw t2, 248(s2) # SRAM_0[62] = X	twiddle preload for stage_1~4	delay
87	sw t3, 252(s2) # SRAM_0[63] = Y	twiddle preload for stage_1~4	delay
88	twiddle preload for stage_1~4	delay	lw t0, 0(s2) # load SRAM0[0]
89	twiddle preload for stage_1~4	lw t2, 4(s4) # t2 = dpath_reg_0 (X = A + WB)	lw t1, 8(s2) # load SRAM0[2]
90	delay	lw t3, 4(s5) # t3 = dpath_reg_1 (Y = A - WB)	sw t0, 0(s4) # store to datapath input 0
91	lw t0, 0(s2) # load SRAM0[0]	sw t2, 248(s2) # SRAM_0[62] = X	sw t1, 0(s5) # store to datapath input 1
92	lw t1, 24(s2) # load SRAM0[6]	sw t3, 252(s2) # SRAM_0[63] = Y	sw s7, 0(s6) # store twiddle factor W[0]
93	sw s8, 0(s4) # store preloaded SRAM0[4]	delay	lw s8, 16(s2) # preload SRAM0[4] for next butterfly
94	sw t1, 0(s5)	lw t0, 0(s2) # load SRAM0[0]	lw t2, 4(s4) # load result X from datapath
95	sw s7, 0(s6)	lw t1, 24(s2) # load SRAM0[6]	lw t3, 4(s5) # load result Y from datapath
96	lw s8, 32(s2) # preload SRAM0[8]	sw s8, 0(s4) # store preloaded SRAM0[4]	sw t2, 0(s2) # store X to SRAM0[0]
97	lw t2, 4(s4)	sw t1, 0(s5)	sw t3, 8(s2) # store Y to SRAM0[2]
98	lw t3, 4(s5)	sw s7, 0(s6)	lw t1, 24(s2) # load SRAM0[6]
99	sw t2, 16(s2) # store to SRAM0[4]	lw s8, 32(s2) # preload SRAM0[8]	sw s8, 0(s4) # store preloaded SRAM0[4]
100	sw t3, 24(s2) # store to SRAM0[6]	lw t2, 4(s4)	sw t1, 0(s5)
101	lw t1, 24(s2) # load SRAM0[6]	lw t3, 4(s5)	sw s7, 0(s6)
102	sw s8, 0(s4) # store preloaded SRAM0[4]	sw t2, 16(s2) # store to SRAM0[4]	lw s8, 32(s2) # preload SRAM0[8]
103	sw t1, 0(s5)	sw t3, 24(s2) # store to SRAM0[6]	lw t2, 4(s4)
104	sw s7, 0(s6)	lw t1, 24(s2) # load SRAM0[6]	lw t3, 4(s5)
105	lw s8, 32(s2) # preload SRAM0[8]	sw s8, 0(s4) # store preloaded SRAM0[4]	sw t2, 16(s2) # store to SRAM0[4]
106	lw t2, 4(s4)	sw t1, 0(s5)	sw t3, 24(s2) # store to SRAM0[6]
107	lw t3, 4(s5)	sw s7, 0(s6)	lw t1, 24(s2) # load SRAM0[6]
108	sw t2, 16(s2) # store to SRAM0[4]	lw s8, 32(s2) # preload SRAM0[8]	sw s8, 0(s4) # store preloaded SRAM0[4]
109	sw t3, 24(s2) # store to SRAM0[6]	lw t2, 4(s4)	sw t1, 0(s5)

Strategy Overview

Strategy_1

SRAM occupancy를 192 → 64로 줄이기 위해 한 개의 set를 3개의 core가 동시에 bf연산 수행

Strategy_2

1개의 bf를 3의 배수(9, 15) clk-cycle로 맞춰 bf연산 수행을 규칙적으로 만듦으로써, 1R1W 준수 여부를 효율적으로 관리

Strategy_3

Dpath_reg의 연산시간(1 clk-cycle)에 이전 bf에 대한 preload를 진행

Strategy_4

stage_5에서 delayed store를 진행함으로써, 1R1W 준수

Strategy_5

Dpath_reg의 연산시간(1 clk-cycle)에 stage_1~4에 대한 Twiddle factor lw

Results

Before

```
Validating bit-accurate output...  
  - Total mismatches: 0 / 192  
Checking SRAM occupancy...  
  - SRAM 2: occupancy = 64  
  - SRAM 3: occupancy = 64  
  - SRAM 5: occupancy = 64  
Test passed. Ending simulation.  
#finish called from file "../..tb/tb_mem_IO.v", line 132.  
#finish at simulation time 4394000  
Simulation complete, time is 4394000 ps.
```

소요시간 16000ps(8 clk-cycle)감소

After

```
Output count has reached the total number of samples.  
Validating bit-accurate output...  
  - Total mismatches: 0 / 192  
Checking SRAM occupancy...  
  - SRAM 2: occupancy = 64  
Test passed. Ending simulation.  
#finish called from file "../..tb/tb_mem_IO.v", line 132.  
#finish at simulation time 4378000  
Simulation complete, time is 4378000 ps.
```


Results

Before

```
Validating bit-accurate output...  
  - Total mismatches: 0 / 192  
Checking SRAM occupancy...  
  - SRAM 2: occupancy = 64  
  - SRAM 3: occupancy = 64  
  - SRAM 5: occupancy = 64  
Test passed. Ending simulation.  
#finish called from file "../..tb/tb_mem_IO.v", line 132.  
#finish at simulation time 4394000  
Simulation complete, time is 4394000 ps.
```

SRAM occupancy 1/3배로 감소

After

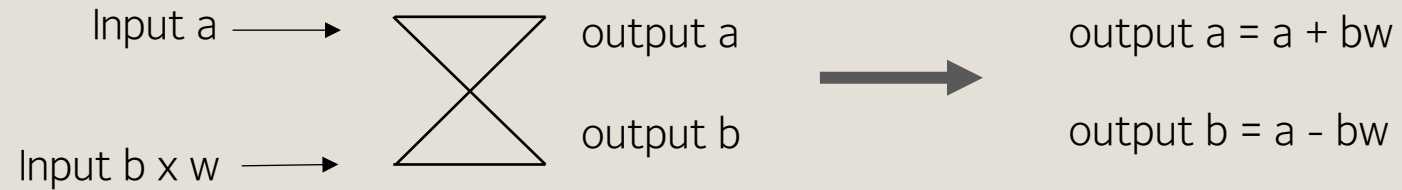
```
Output count has reached the total number of samples.  
Validating bit-accurate output...  
  - Total mismatches: 0 / 192  
Checking SRAM occupancy...  
  - SRAM 2: occupancy = 64  
Test passed. Ending simulation.  
#finish called from file "../..tb/tb_mem_IO.v", line 132.  
#finish at simulation time 4378000  
Simulation complete, time is 4378000 ps.
```

Roofline model

=> Ideal model

=> 1cycle 당 1개의 Butterfly 수행

<한 개의 Butterfly당 수행하는 곱셈 덧셈 개수 구하기 (by RTL code)>



i) bw 17H

실수 $b \times$ 실수 w
 실수 $b \times$ 허수 w
 허수 $b \times$ 실수 w
 허수 $b \times$ 허수 w } 4번

iii) $a + bw \Rightarrow$ 실수 $a +$ 실수 bw
허수 $a +$ 허수 bw

$a - bw \Rightarrow$ 실수 $a -$ 실수 bw
허수 $b -$ 허수 bw

} 4번

ii) 결과 bw를 Quantization 하기 위한 덧셈 → 2번

1 Butterfly당 총 10FLOPs

1) Peak Performance

Peak Performance = (한 주기동안 최대로 수행하는 FLOPs 수) x (Frequency)

$$= (10\text{FLOPs/cycle}) \times (0.5\text{G cycle/s}) = 5\text{GFLOPs}$$

$\rightarrow 1/2000\text{ps} = 0.5\text{GHz}$

2) Operational Intensity

O.I = (total computation amount [FLOPs] / (total memory traffic [Bytes])) = 0.6 [FLOPs/Bytes]

<Reuse 고려>

total FLOPs = 3set x 6 stage x 32 B.f x 10FLOPs => 5760 FLOPs
total memory traffic = input A/B & twiddle factor & output => 9576 Byte

O.I = 5760 [FLOPs] / 9636 [Byte] = 0.6 [FLOPs/Bytes]

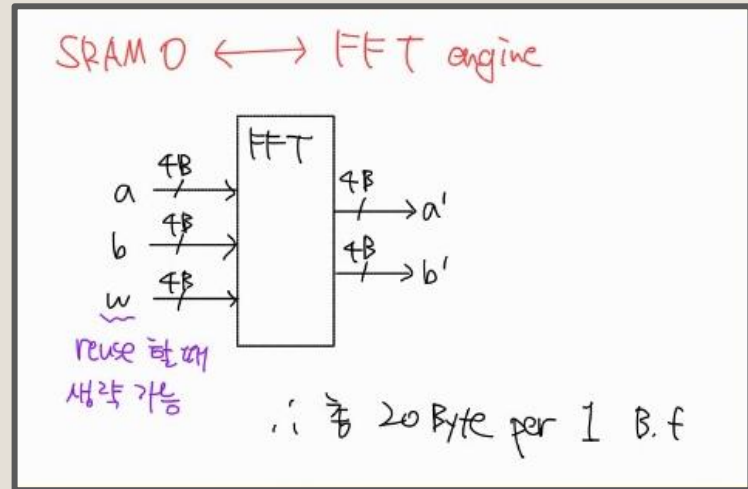
Stage 0 : w load 1회

Stage 1~4 : w load x

Stage 5 : w load 32회

$\rightarrow$ w load 34회 Reuse : 157번
20 Byte 16 Byte

$\rightarrow$ Total = (34 x 20 + 157 x 16) x 3 = 9576 Byte



Bandwidth = (total memory traffic) / (total execution time) = 20 Byte / 2000 ps = 10G Byte / sec → B.f 하나 기준

Attainable Performance = $O.I \times BW$ = 10G Byte / sec x 0.6 FLOPs / Byte = 6GFLOPS

<Roofline Model (Graph)>

